# IEEE Student Branch - PRIORITY-FOCUSED 3-Month Plan

**Focus: Events, Registration, Attendance, Emails - The Core Workflow Team: 3 people × 2-3 hrs/week = 6-9 hrs/week total**

---

## 🎯 CORE INSIGHT

**What you ACTUALLY need**: A system that handles the repeating event workflow:

1. Create event → 2. Students register → 3. Track attendance → 4. Email communications

**Everything else is secondary.**

---

## 📋 FEATURE PRIORITY MATRIX

**TIER 1: CRITICAL (Must have in 3 months)**

These enable your core workflow:

1. ✅ **Event Management** (create, view, register)
2. ✅ **Attendance System** (mark, export CSV, email attendees)
3. ✅ **Email System** (bulk emails with smart filters)
4. ✅ **4-Role System** (Student, Member, Board, ExCom)
5. ✅ **Season Management** (create seasons, switch between them)

**TIER 2: IMPORTANT (Build after core works)**

Support features:

6. ⭐ **Form Builder** (for registration forms, surveys)
7. ⭐ **File Uploads** (resumes, certificates)
8. ⭐ **User Profiles**

**TIER 3: NICE TO HAVE (Can be basic/simple)**

9. 🔹 **Committee Pages** (just header + brief description)
10. 🔹 **Analytics** (basic stats)

**TIER 4: POST-LAUNCH (Months 4-6)**

11. ⏳ Blog/News system
12. ⏳ Assignments
13. ⏳ Google OAuth
14. ⏳ Advanced analytics

---

## 🚀 STRATEGIC 3-MONTH PLAN

**Core Strategy**

**Month 1**: Build the event workflow (80% of your needs) **Month 2**: Add seasons + email intelligence **Month 3**: Form builder + polish + deploy

---

## 📅 MONTH 1: EVENT WORKFLOW (Weeks 1-4)

**Goal**: Create → Register → Attend → Email workflow working **Hours**: 24-36 total

**Week 1: Foundation (6-9 hrs)**

**You (3 hrs)**:

- [ ] Setup GitHub repos
- [ ] Initialize MERN stack
- [ ] Design database schema (focus on Users, Events, Attendance, Seasons)
- [ ] Create development guide

**Backend (2-3 hrs)**:

- [ ] Express server + MongoDB connection
- [ ] User model with 4 roles + season membership
- [ ] Season model (basic)
- [ ] Auth endpoints (register, login, JWT)

**Frontend (2-3 hrs)**:

- [ ] React setup with Tailwind/Material-UI
- [ ] Login/Register pages
- [ ] Auth context
- [ ] Route structure

---

**Week 2: Events Core (6-9 hrs)**

**Backend (3 hrs)**:

- [ ] Event model (linked to season)
- [ ] Create event API (Member+)
- [ ] Get events API (filter by season, upcoming/past)
- [ ] Register for event API
- [ ] Unregister API

**Frontend (3 hrs)**:

- [ ] Event list page (cards with register button)
- [ ] Event detail page
- [ ] Create event form (Member+)
- [ ] Registration system

**You (2 hrs)**:

- [ ] Integration testing
- [ ] Fix critical bugs

---

**Week 3: Attendance System (6-9 hrs)**

**Backend (3 hrs)**:

- [ ] Attendance model (event + user + season)
- [ ] Mark attendance API (bulk and single)
- [ ] Get attendance for event
- [ ] Export attendance CSV endpoint
- [ ] Get registered vs attended stats

**Frontend (3 hrs)**:

- [ ] Attendance marking page (checkboxes for registered users)
- [ ] Bulk select/unselect
- [ ] Attendance list view
- [ ] Download CSV button
- [ ] Stats display (X/Y attended)

**You (2 hrs)**:

- [ ] Test full workflow: create event → students register → mark attendance → export

---

**Week 4: Basic Emails (6-9 hrs)**

**You (3 hrs)**:

- [ ] Integrate existing email code
- [ ] Setup Nodemailer with Gmail
- [ ] Create email templates (event reminder, welcome)

**Backend (3 hrs)**:

- [ ] Send email API (to array of emails)
- [ ] Send to event attendees API
- [ ] Send to event registrants API
- [ ] Send to all users by role API
- [ ] Email log model

**Frontend (2 hrs)**:

- [ ] Email composer (simple)
- [ ] Quick actions: "Email all attendees", "Email registrants"
- [ ] Send email button

**Milestone**: 🎉 Core workflow complete!

---

## 📅 MONTH 2: INTELLIGENCE (Weeks 5-8)

**Goal**: Season management + Smart email filters **Hours**: 24-36 total

**Week 5: Season Management (6-9 hrs)**

**Backend (3 hrs)**:

- [ ] Season CRUD APIs
- [ ] Create season (ExCom only)
- [ ] Set active season
- [ ] Archive season
- [ ] User season membership management

**Frontend (3 hrs)**:

- [ ] Season selector dropdown (global)
- [ ] Create new season modal (ExCom)
- [ ] Season management page (list, archive)

- [ ] All views filtered by active season

**You (2 hrs)**:

- [ ] Test season switching
- [ ] Ensure all data scoped to seasons

---

**Week 6: Advanced Email Filters (6-9 hrs)**

**Backend (4 hrs)**:

- [ ] Dynamic filter API:
  - By role (student, member, board, excom)
  - By attendance (attended event X, didn't attend)
  - By registration (registered but didn't attend)
  - By season
  - Combinations (AND/OR logic)
- [ ] Preview recipients API (count + emails)

**Frontend (3 hrs)**:

- [ ] Advanced email composer
- [ ] Filter builder UI:
  - Checkboxes for roles
  - Dropdown for events (attended/not attended)
  - Season filter
  - Custom email list upload
- [ ] Preview recipients modal
- [ ] Send confirmation dialog

**You (1 hr)**:

- [ ] Test complex filters

---

**Week 7: Role System Polish (6-9 hrs)**

**Backend (3 hrs)**:

- [ ] Role-based middleware (protect routes)

- [ ] User management APIs:
  - Get all users (ExCom)
  - Update user role (ExCom)
  - Promote users between seasons
- [ ] Permission checks on all endpoints

**Frontend (3 hrs)**:

- [ ] User management page (ExCom only)
- [ ] Role badges throughout UI
- [ ] Permission-based UI rendering
- [ ] User profile page

**You (2 hrs)**:

- [ ] Security audit
- [ ] Test all role permissions

---

**Week 8: File Uploads (6-9 hrs)**

**Backend (3 hrs)**:

- [ ] Setup Cloudinary SDK
- [ ] File upload middleware (validation, size limits)
- [ ] Upload APIs:
  - Profile picture
  - Event poster
  - Resume/certificates
- [ ] File metadata storage

**Frontend (3 hrs)**:

- [ ] File upload component (drag & drop)
- [ ] Image preview
- [ ] Upload progress bar
- [ ] Display uploaded files

**You (2 hrs)**:

- [ ] Test uploads
- [ ] Cloudinary configuration

**Milestone**: 🎉 Smart system complete!

---

## 📅 MONTH 3: FORMS & LAUNCH (Weeks 9-12)

**Goal**: Form builder + Deploy + Launch **Hours**: 24-36 total

**Week 9: Form Builder Foundation (6-9 hrs)**

**Backend (3 hrs)**:

- [ ] Form model (dynamic fields structure)
- [ ] Form CRUD APIs
- [ ] Form submission model
- [ ] Submit form API

**Frontend (4 hrs)**:

- [ ] Form builder page (drag-drop or simple list)
- [ ] Field types: text, email, number, textarea, select, radio, checkbox, date
- [ ] Add/remove fields
- [ ] Reorder fields

**You (1 hr)**:

- [ ] Integration testing

---

**Week 10: Form Responses & Committee (6-9 hrs)**

**Backend (2 hrs)**:

- [ ] Get form submissions API
- [ ] Export submissions CSV
- [ ] Committee model (simple)
- [ ] Committee CRUD APIs

**Frontend (4 hrs)**:

- [ ] Public form fill page
- [ ] View submissions page
- [ ] Export responses button
- [ ] **Committee page** (simple: logo, name, description, member list)

**You (2 hrs)**:

- [ ] Test forms end-to-end
- [ ] Create committee pages

---

**Week 11: Deployment (6-9 hrs)**

**You (4 hrs)**:

- [ ] Buy and configure domain
- [ ] MongoDB Atlas production setup
- [ ] Deploy backend to Render
- [ ] Deploy frontend to Vercel
- [ ] SSL + DNS configuration
- [ ] Environment variables

**Backend (2 hrs)**:

- [ ] Production build testing
- [ ] Security headers (Helmet)
- [ ] Rate limiting

**Frontend (2 hrs)**:

- [ ] Production build
- [ ] Environment configs
- [ ] Fix production bugs

---

**Week 12: Testing & Launch (6-9 hrs)**

**Everyone (2-3 hrs each)**:

- [ ] Full system testing
- [ ] Mobile responsiveness check
- [ ] Loading states and error handling
- [ ] Create admin guide
- [ ] Setup error tracking (Sentry)

**Launch Plan**:

- [ ] Beta test with 10-20 users (days 1-3)
- [ ] Fix critical bugs (days 4-5)
- [ ] Official launch (day 6)
- [ ] Monitor and support (day 7)

**Milestone**: 🚀 LIVE!

---

## 🗄️ SIMPLIFIED DATABASE MODELS

### Priority Models (Build these first)

```javascript
```

```
// 1. Season (Week 1)
{
  name: "2024-2025",
  startDate: Date,
  endDate: Date,
  isActive: Boolean,
  status: "active" | "archived"
}

// 2. User (Week 1)
{
  email: String,
  password: String,
  name: String,
  seasonMemberships: [{
    season: ObjectId,
    role: "student" | "member" | "board_member" | "excom",
    joinedAt: Date
  }],
  currentSeason: ObjectId,
  currentRole: String,
  profileImage: String, // Cloudinary (Week 8)
  resume: String, // Cloudinary (Week 8)
}

// 3. Event (Week 2)
{
  title: String,
  description: String,
  season: ObjectId,
  date: Date,
  location: String,
  poster: String, // Cloudinary (Week 8)
  maxAttendees: Number,
  registeredUsers: [{user: ObjectId, registeredAt: Date}],
  status: "upcoming" | "completed",
  createdBy: ObjectId
}

// 4. Attendance (Week 3)
{
  event: ObjectId,
  user: ObjectId,
```

```
  season: ObjectId,
  markedAt: Date,
  markedBy: ObjectId
}

// 5. EmailLog (Week 4)
{
  subject: String,
  body: String,
  recipientCount: Number,
  recipientEmails: [String],
  filters: Object,
  sentBy: ObjectId,
  sentAt: Date
}

// 6. Form (Week 9)
{
  title: String,
  season: ObjectId,
  fields: [{
    type: String,
    label: String,
    required: Boolean,
    options: [String]
  }],
  isActive: Boolean,
  createdBy: ObjectId
}

// 7. FormSubmission (Week 9)
{
  form: ObjectId,
  user: ObjectId,
  responses: [{fieldId: String, value: Mixed}],
  submittedAt: Date
}

// 8. Committee (Week 10 - SIMPLE)
{
  name: String,
  description: String,
  season: ObjectId,
  logo: String,
```

```
  members: [{user: ObjectId, position: String}]
}
```

---

## 🎯 WHAT YOU'LL HAVE AFTER 3 MONTHS

### ✅ Complete Event Workflow

1. **Create Event** (Member/Board/ExCom)
   - Title, description, date, location, poster
   - Set max attendees
   - Link to current season
2. **Student Registration**
   - Browse events
   - Register/unregister
   - See registration status
3. **Attendance Tracking**
   - Mark attendance (checkboxes)
   - Bulk select
   - Export CSV with stats
   - Email attendees directly
4. **Smart Email System**
   - Email all students/members/board/excom
   - Email event attendees
   - Email event registrants who didn't attend
   - Email by custom filters
   - Preview recipients before sending
   - Track email history

### ✅ Season Management

- ExCom creates new seasons
- All data scoped to seasons

- Switch between seasons (ExCom view)
- Archive old seasons

✅ **4-Role System**

- **Student**: View, register for events
- **Member**: + Create events, mark attendance, send emails
- **Board Member**: + Manage users, create forms
- **ExCom**: + Season management, full analytics

✅ **Form Builder**

- Create dynamic forms
- Students fill forms
- View submissions
- Export responses as CSV

✅ **File Uploads**

- Profile pictures
- Event posters
- User resumes/certificates

✅ **Simple Committee Pages**

- Name, logo, description
- List of members with positions

---

## 🔥 CRITICAL SUCCESS STRATEGIES

**1. Work Parallel (MOST IMPORTANT)**

Never wait for each other:

**Week 2 Example**:

- Backend: Builds Event APIs
- Frontend: Builds Event UI using MOCK data
- You: Reviews database design

- **Friday**: Backend done → Frontend connects real APIs → Merge

**2. Use Templates & Libraries**

Don't build from scratch:

- **Form Builder**: Use `react-form-builder2` or `Survey.js`
- **Rich Text**: Use `React Quill`
- **UI Components**: Material-UI or shadcn/ui
- **File Upload**: Use Cloudinary's upload widget
- **Email**: Copy your existing email app code

**3. MVP Everything**

First version of every feature is SIMPLE:

- Attendance? Just checkboxes (QR codes later)
- Email? Just text body (rich text later)
- Form builder? Simple fields only (conditional logic later)
- Committee page? Just text + image (fancy layout later)

**4. Deploy Early & Often**

- Week 3: Deploy to staging (even if broken)
- Week 6: Show to IEEE board for feedback
- Week 9: Beta test with 10 users
- Week 12: Full launch

**5. Communication Protocol**

With only 2-3 hrs/week, communication is CRITICAL:

**Monday**:

- Check GitHub Projects for your task
- Comment: "Starting task X"

**Mid-week**:

- Push progress (even if incomplete)
- Flag blockers immediately

**Friday**:

- Complete task
- Create PR
- Review others' PRs
- Merge approved PRs

**No meetings, no calls, just GitHub + Discord.**

---

## 📊 TASK DISTRIBUTION

**Backend Developer (Focus: APIs & Logic)**

- All database models
- All REST APIs
- Authentication & authorization
- Email service integration
- Season logic
- CSV export functionality

**Frontend Developer (Focus: UI Only)**

- All pages and components
- Form handling
- API integration (you provide endpoints)
- Styling (Tailwind/Material-UI)
- Responsive design

**You (Full Stack - Architect & Integrator)**

- Database schema design
- Complex features (email filters, season management)
- Integration of frontend + backend
- Deployment & DevOps
- Security

- Code reviews
- Unblocking team members

---

## ⚠️ RISKS & SOLUTIONS

**Risk: "Form builder too complex"**

**Solution**: Use `Survey.js` library instead of building from scratch **Time saved**: 10-15 hours

**Risk: "Email system too complex"**

**Solution**:

- Week 4: Basic email (send to list)
- Week 6: Add filters incrementally
- Don't over-engineer

**Risk: "Season management confusing"**

**Solution**:

- Make ONE season active at a time
- All queries automatically filter by active season
- ExCom can switch seasons in UI only

**Risk: "Team member gets stuck"**

**Solution**:

- Timebox tasks to 3 hours max
- If stuck after 2 hours → Ask for help in Discord
- You jump in to unblock

**Risk: "Won't finish in 3 months"**

**Acceptance**:

- That's okay! Launch what works
- Blog/assignments can come later (Month 4-6)
- Users will forgive missing features if core works

---

## 🎯 MINIMUM VIABLE LAUNCH CHECKLIST

Must work on launch day:

- [ ] Users can register and login
- [ ] Members can create events
- [ ] Students can register for events
- [ ] Members can mark attendance
- [ ] Can export attendance CSV
- [ ] Can email groups (students, attendees, etc.)
- [ ] ExCom can manage seasons
- [ ] Basic forms work
- [ ] File uploads work
- [ ] Site is live on custom domain
- [ ] Mobile responsive

Can be buggy but must exist:

- [ ] User profiles
- [ ] Committee pages (simple)
- [ ] Role management

Can wait for post-launch:

- Blog/news system (Month 4)
- Assignments (Month 5)
- Advanced analytics (Month 6)
- QR code attendance (Month 6)

---

## 💡 WEEK 1 KICKOFF (START HERE)

**Day 1 (1 hour - Everyone)**

**You**:

- [ ] Create GitHub organization
- [ ] Create repos: `ieee-webapp-backend`, `ieee-webapp-frontend`
- [ ] Setup GitHub Projects with Month 1 tasks
- [ ] Share this plan with team

**Backend**:

- [ ] Clone backend repo
- [ ] Install Node.js, MongoDB Compass
- [ ] Read setup guide

**Frontend**:

- [ ] Clone frontend repo
- [ ] Install Node.js
- [ ] Read setup guide

**Day 2-7 (Rest of Week 1)**

Follow Week 1 tasks from Month 1 plan above.

**Goal**: By end of Week 1, you have:

- Working login/register
- GitHub workflow established
- Everyone knows their role

---

## 🚀 FINAL WORDS

**You've prioritized correctly**: Events + Attendance + Emails = 80% of your actual usage.

With 72-108 hours over 3 months, this plan is **aggressive but achievable** because:

- ✅ You're building what you ACTUALLY need
- ✅ Using libraries (not building from scratch)
- ✅ Working in parallel
- ✅ Launching MVP, iterating later

**The secret**: Ship a working 80% product in 3 months, not a perfect 100% product in 12 months.

**Remember**:

- Blog can wait (post-launch)
- Assignments can wait (post-launch)
- Advanced analytics can wait (post-launch)

But **events, attendance, and emails cannot wait** - that's your daily workflow.

You've got this! 🚀

Start with Week 1, and let me know if you need help with database schemas, API endpoints, or anything else!